

Salesforce Developer Bridge Program

Day 4 – Self Learning Track Report

Placement Management System — Lightning Web Components

Student Details

Name: Sudharshan Kuppli

Project: Placement Management System

Day: Day 4

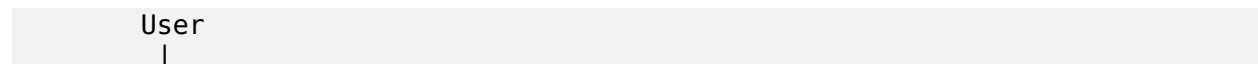
Technology: Lightning Web Components (LWC)

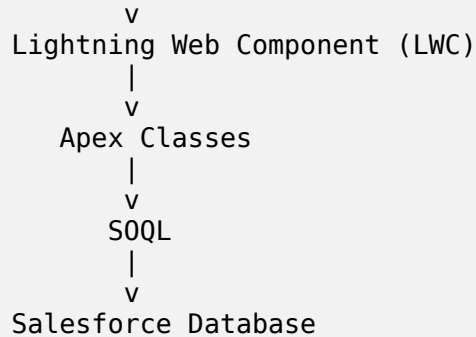
Day Objective

The objective of Day 4 was to move from the backend work completed in earlier sprints — data modeling, SOQL, Apex triggers, and automated business rules — into the front end of the Placement Management System. Specifically, the goal was to learn Lightning Web Components (LWC), Salesforce's modern user interface framework, and use it to build the first screen a real user (the Placement Officer or a student) would actually interact with. This included understanding the structure of an LWC, creating and deploying the first component, implementing data binding, handling user interactions through JavaScript, and assembling a small working dashboard.

Architecture Overview

Before writing any UI code, it helps to see where a Lightning Web Component sits in the overall Salesforce application. The architecture followed throughout Day 4 is:





Day 4 focuses entirely on the top layer of this diagram — the LWC. The component displays information and captures user actions (like button clicks), but on Day 4 it does not yet call Apex or query the database directly; the values used (student name, roll number, department, company/job counts) are hard-coded in JavaScript. This is a deliberate, incremental approach: build and prove the UI layer first, then connect it to Apex and SOQL in a later sprint. Understanding this separation up front makes it clear why LWC, Apex, and SOQL are treated as three distinct layers, each with its own responsibility, rather than one large block of mixed logic.

Learning Objectives

By the end of Day 4, the goal was to be able to:

- Explain what a Lightning Web Component is and why Salesforce introduced it.
- Describe the three files that make up every LWC and the responsibility of each.
- Create, deploy, and activate a new LWC using the Salesforce CLI and Lightning App Builder.
- Use data binding to display JavaScript variables in the HTML template.
- Handle a button click event and update the UI in response.
- Manage simple UI state (such as an application status) entirely on the client side.
- Assemble several small pieces of UI into one small working dashboard.

Task 1 - Understanding Lightning Web Components

Description

Learned that Lightning Web Components (LWC) is Salesforce's modern UI framework, built using standard web technologies — HTML, JavaScript, and CSS — instead of a Salesforce-proprietary markup language. Understood the architecture: User → LWC → Apex → SOQL → Salesforce Database.

Why This Task Was Performed

Every Salesforce application eventually needs a screen a real person can use. Before this task, all of the work in the Placement Management System project lived entirely on the server side — objects, fields, triggers, and validation rules that a user never sees directly. Task 1 exists to introduce the layer that finally makes the application usable: the user interface.

Salesforce Concepts Involved

LWC is built on top of Web Components, a set of modern browser standards, which is precisely why it can reuse ordinary HTML, JavaScript, and CSS skills instead of requiring a Salesforce-only syntax. This is also the reason LWC is now preferred over the older Aura Components framework: Aura predates these browser standards and carries more Salesforce-specific overhead, while LWC is lighter, faster, and closer to how the rest of the web is built.

Business Purpose

For the Placement Management System, this means the Placement Officer and students will eventually interact with a fast, modern, native-feeling screen — not a generic list of database records. Learning LWC is what allows the business rules already built in Apex to become something people can actually use day to day.

Expected Output

No component was built in this task — the outcome was conceptual: a clear understanding of what LWC is and where it fits in the User → LWC → Apex → SOQL → Salesforce Database flow, ready to be put into practice in Task 2 onward.

Learning Outcome

- Understood LWC architecture.
- Learned frontend development in Salesforce.

Task 2 – Understanding LWC Structure

Description

Every LWC contains three files, each with a distinct responsibility:

- placementHome.html — the template, responsible for layout, text, and structure.
- placementHome.js — the JavaScript controller, responsible for variables, logic, and events.
- placementHome.js-meta.xml — the configuration file, responsible for exposing the component and declaring where it can be used.

Why This Task Was Performed

Before creating a component, it is important to know what each file is for. Without this understanding, a beginner might try to put layout markup inside the JavaScript file, or configuration inside the HTML file. Learning the three-file structure first avoids that confusion later.

Salesforce Concepts Involved

The .js-meta.xml file is the piece unique to Salesforce — it has no equivalent in plain web development. It tells the platform which API version the component targets, whether it is exposed for use outside its own bundle, and which page types (App Page, Home Page, Record Page) it is allowed to appear on.

A short explanation before the code: the metadata file below is what makes placementHome selectable inside Lightning App Builder — without isExposed set to true and at least one target listed, the component would exist in the org but never appear as an option when building a page.

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>67.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
    <target>lightning__RecordPage</target>
  </targets>
</LightningComponentBundle>
```

What this code does: it declares the component's API version (67.0), marks it as isExposed so it can be added through the UI rather than only referenced by other components, and lists three target page types — meaning placementHome can be dropped onto an App Page, a Home Page, or a Record Page in Lightning App Builder.

Business Purpose

This configuration step is what actually gives the Placement Officer the flexibility to place the component wherever it is most useful — for example, on a general Home Page for daily use, or on a specific Record Page tied to a particular job posting.

Expected Output

A correctly structured three-file bundle inside force-app/main/default/lwc/placementHome, visible in the project explorer and ready to be edited.

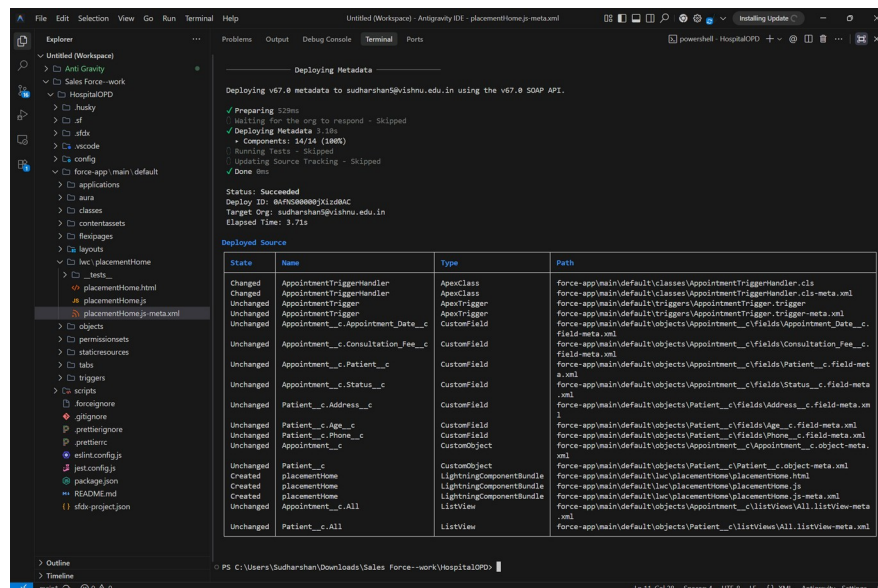


Figure 1: The placementHome LWC Bundle — html, js, and js-meta.xml Files in the Project Explorer

Best Practices

- Keep the .js-meta.xml file's target list limited to the page types the component is actually meant to appear on, rather than exposing it everywhere by default.
- Match the apiVersion in the metadata file to the org's current supported version.

Learning Outcome

- Learned HTML structure.
 - Learned JavaScript logic.
 - Understood Meta XML configuration.
-

Task 3 – Creating the First LWC

Description

Created placementHome using the Salesforce CLI and displayed a welcome message, then deployed it and added it to a Lightning App Page.

Why This Task Was Performed

This is the first hands-on proof that the concepts from Tasks 1 and 2 actually work end to end: writing a component, deploying it to a real org, and seeing it render on an actual page.

Salesforce Concepts Involved

The HTML template below uses lightning-card, a pre-built Lightning Design System (SLDS) component that gives the component a consistent header and bordered container for free, and slds-p-around_medium, an SLDS utility class that adds consistent spacing without any custom CSS.

```
<template>
  <lightning-card title="Placement Portal">
    <div class="slds-p-around_medium">
      <h1 class="slds-text-heading_large">
        Welcome to Vishnu Placement Portal
      </h1>
    </div>
  </lightning-card>
</template>
```

What this code does: it wraps the component's content in a lightning-card titled "Placement Portal," adds medium padding around the inner content, and displays a large heading that reads "Welcome to Vishnu Placement Portal" — the very first thing a user sees when the component loads.

Business Purpose

A working, deployed, branded welcome screen is a small but meaningful milestone: it is the first piece of the Placement Management System that is actually visible and usable by a real person, rather than something only visible in Setup or the Developer Console.

Expected Output

A successful CLI deployment, followed by the component appearing as an option inside Lightning App Builder and rendering the welcome heading once added to the page.

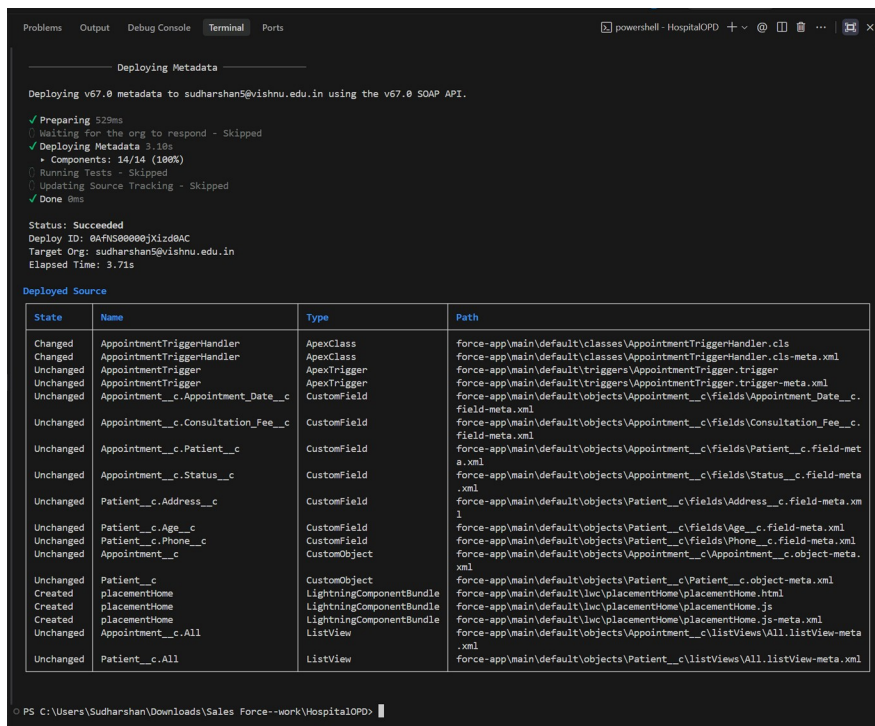


Figure 2: Successful CLI Deployment of the placementHome Component

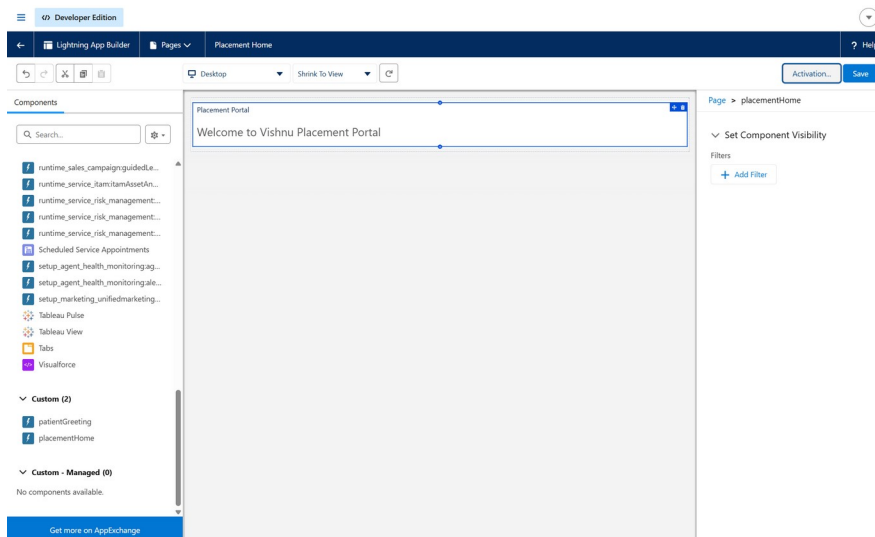


Figure 3: placementHome Added to the Placement Home Page in Lightning App Builder

Best Practices

- Deploy early and often in small steps, rather than writing the entire component before the first deployment.
- Use SLDS components and utility classes (lightning-card, slds-p-around_medium) instead of custom CSS wherever possible, for a consistent Salesforce look and feel.

Learning Outcome

- Created my first LWC.
- Learned deployment and activation.

Task 4 - Displaying Student Information

Description

Added JavaScript variables for student name, roll number, and department, and displayed them in the template using data binding.

Why This Task Was Performed

A static welcome message proves the component renders, but real applications display data. This task introduces the core mechanism LWC uses to get information from JavaScript onto the screen.

Salesforce Concepts Involved

The class below extends `LightningElement`, the base class every LWC component class must extend. Declaring a property such as `studentName` directly on the class makes it a reactive property — LWC automatically re-renders any part of the template that references it whenever its value changes.

```
import { LightningElement } from 'lwc';

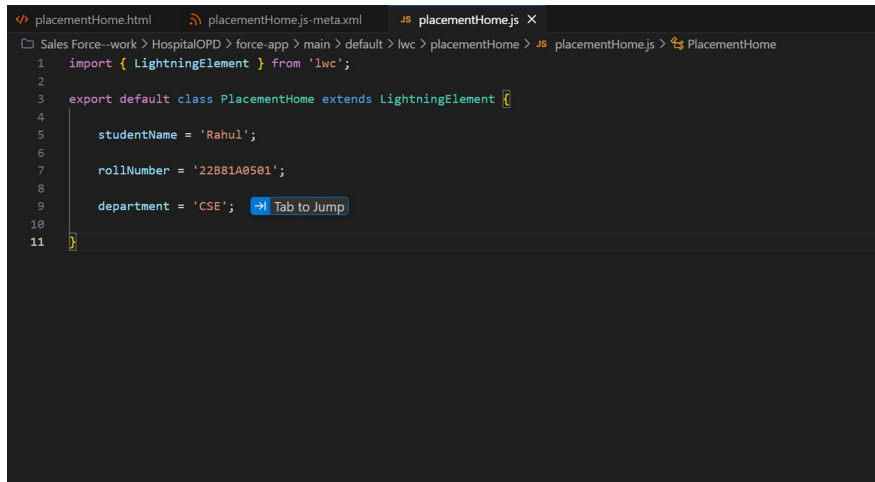
export default class PlacementHome extends LightningElement {
  studentName = 'Rahul';
  rollNumber = '22B81A0501';
  department = 'CSE';
}
```

What this code does: it imports `LightningElement` from the `lwc` module, defines a `PlacementHome` class that extends it, and declares three properties — `studentName`, `rollNumber`, and `department` — each initialised with a hard-coded value that will later come from Apex.

The HTML below binds each of those properties into the page using curly-brace syntax:

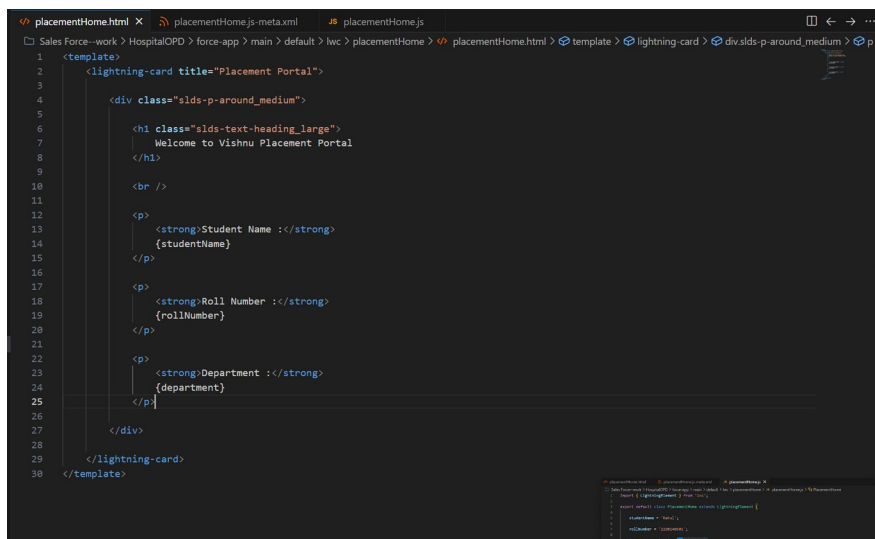
```
<p><strong>Student Name :</strong> {studentName}</p>
<p><strong>Roll Number :</strong> {rollNumber}</p>
<p><strong>Department :</strong> {department}</p>
```

What this code does: each `{propertyName}` expression tells the LWC template engine to insert the current value of that JavaScript property at that exact point in the rendered HTML, and to keep it updated automatically if the property's value ever changes.



```
1 import { LightningElement } from 'lwc';
2
3
4 export default class PlacementHome extends LightningElement {
5     studentName = 'Rahul';
6     rollNumber = '22B81A0501';
7     department = 'CSE';
8 }
9
10
11
```

Figure 4: JavaScript Variables for Student Name, Roll Number, and Department



```
1 <template>
2   <lightning-card title="Placement Portal">
3     <div class="slds-p-around_medium">
4       <h1 class="slds-text-heading_large">
5         Welcome to Vishnu Placement Portal
6       </h1>
7       <br />
8       <p>
9         <strong>Student Name :</strong>
10        {studentName}
11      </p>
12      <p>
13        <strong>Roll Number :</strong>
14        {rollNumber}
15      </p>
16      <p>
17        <strong>Department :</strong>
18        {department}
19      </p>
20    </div>
21  </lightning-card>
22 </template>
```

Figure 5: HTML Template Displaying the Variables using {studentName}, {rollNumber}, {department}

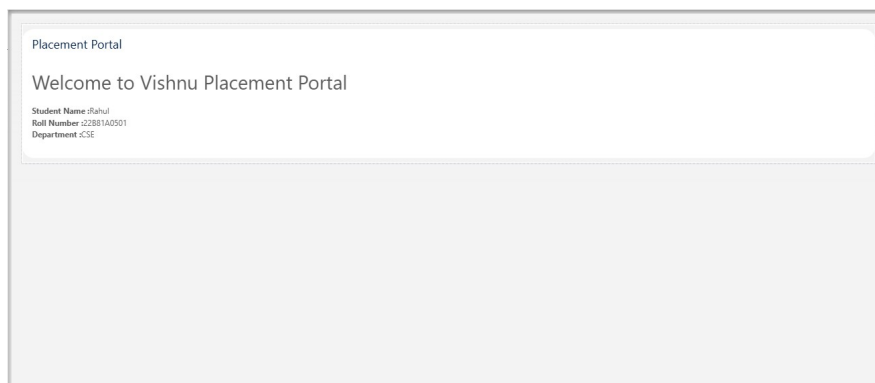


Figure 6: Rendered Component Showing the Bound Student Details

Expected Output

The component renders "Student Name : Rahul", "Roll Number : 22B81A0501", and "Department : CSE" beneath the welcome heading, sourced entirely from the JavaScript class.

Best Practices

- Use descriptive property names (studentName, not sn or x) so the template reads clearly on its own.
- Keep hard-coded placeholder values clearly temporary in your own notes, since they are expected to be replaced by real Apex-sourced data in a later sprint.

Learning Outcome

- Learned data binding using JavaScript variables.

Task 5 – Button Click Event

Description

Added a button labeled "Show Welcome Message" that, when clicked, displays "Welcome to Salesforce Development." on the screen.

Why This Task Was Performed

Displaying static or hard-coded data is only half of what a UI needs to do — the other half is responding to what the user does. This task introduces event handling, the mechanism LWC uses to react to user actions like clicks.

Salesforce Concepts Involved

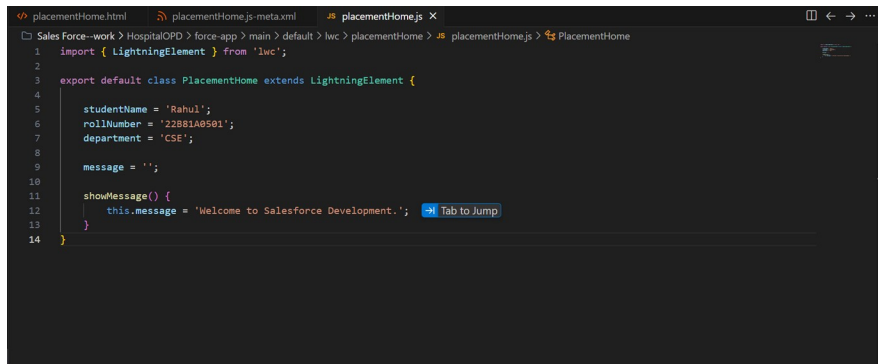
The message property below starts empty and is reassigned inside the showMessage() method using the this keyword, which refers to the current component instance. Because message is a reactive property, updating it inside the method is enough to make the new text appear on screen — no manual DOM manipulation is required.

```
message = '';  
  
showMessage() {  
    this.message = 'Welcome to Salesforce Development.';  
}
```

What this code does: it declares an empty message property, and defines a showMessage() method that, when called, sets this.message to the welcome text — triggering LWC to automatically re-render wherever {message} appears in the template.

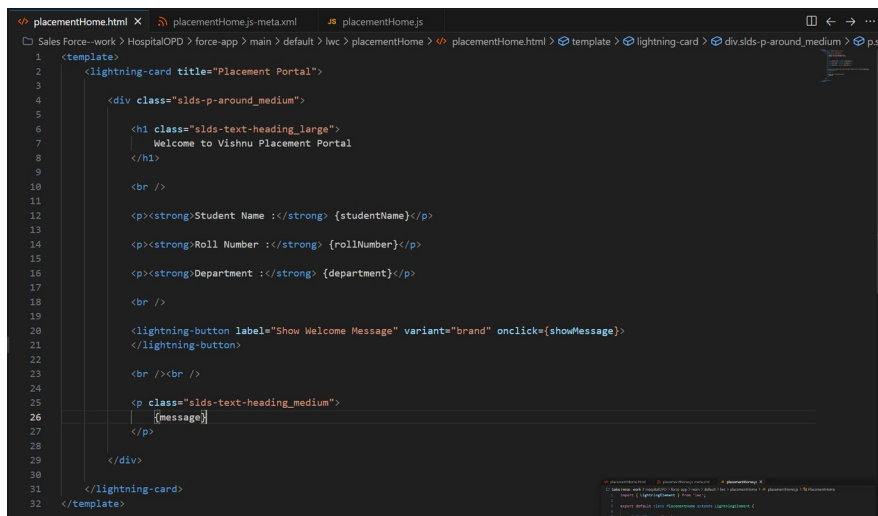
```
<lightning-button  
    label="Show Welcome Message"  
    variant="brand"  
    onclick={showMessage}>  
</lightning-button>  
  
<p>{message}</p>
```

What this code does: it renders a Lightning-styled button labeled "Show Welcome Message," wires its onclick event directly to the showMessage() method using LWC's event-binding syntax, and displays whatever the message property currently holds directly beneath it.



```
1 import { LightningElement } from 'lwc';
2
3 export default class PlacementHome extends LightningElement {
4
5     studentName = 'Rahul';
6     rollNumber = '22881A0501';
7     department = 'CSE';
8
9     message = '';
10
11     showMessage() {
12         this.message = 'Welcome to Salesforce Development.';
13     }
14 }
```

Figure 7: showMessage() Method Added to placementHome.js



```
1 <template>
2     <lightning-card title="Placement Portal">
3
4         <div class="slds-p-around_medium">
5
6             <h1 class="slds-text-heading_large">
7                 Welcome to Vishnu Placement Portal
8             </h1>
9
10            <br />
11
12            <p><strong>Student Name :</strong> {studentName}</p>
13
14            <p><strong>Roll Number :</strong> {rollNumber}</p>
15
16            <p><strong>Department :</strong> {department}</p>
17
18            <br />
19
20            <lightning-button label="Show Welcome Message" variant="brand" onclick={showMessage}>
21            </lightning-button>
22
23            <br /><br />
24
25            <p class="slds-text-heading_medium">
26                {message}
27            </p>
28
29        </div>
30    </lightning-card>
31 </template>
```

Figure 8: lightning-button with onclick={showMessage} in placementHome.html

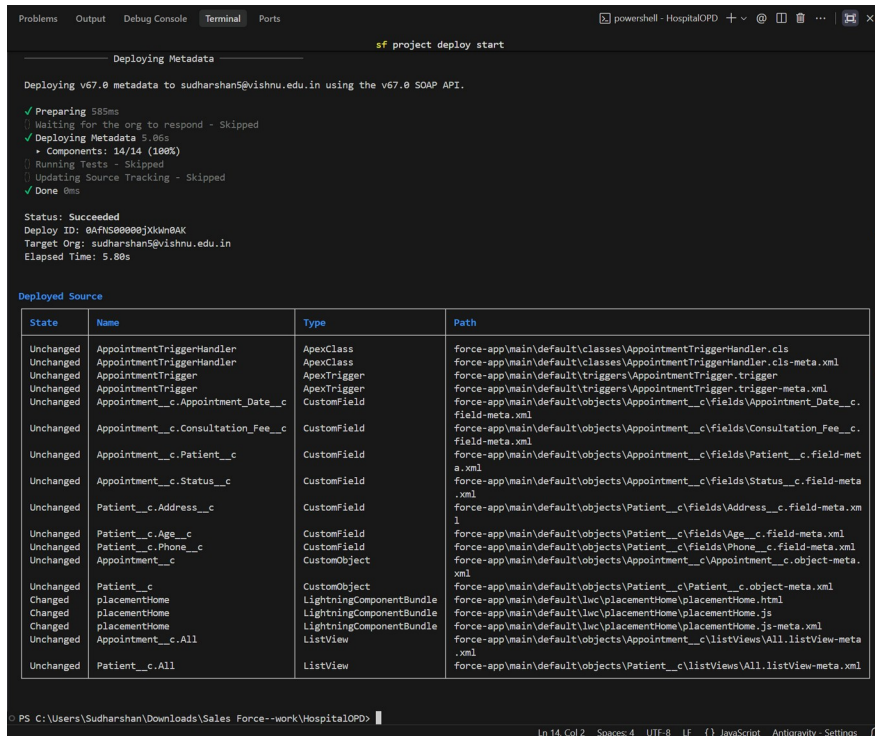


Figure 9: Redeployment of placementHome After Adding the Button Logic

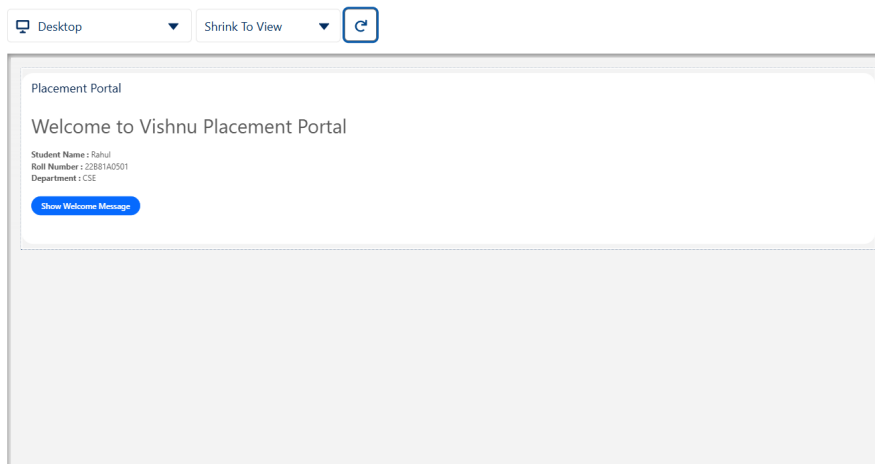


Figure 10: Component in Lightning App Builder Before the Button Is Clicked

Placement Portal

Welcome to Vishnu Placement Portal

Student Name : Rahul
Roll Number : 22B81A0501
Department : CSE

Show Welcome Message

Welcome to Salesforce Development.

Figure 11: Welcome Message Displayed After Clicking "Show Welcome Message"

Expected Output

Before the button is clicked, the paragraph bound to {message} is empty. After clicking "Show Welcome Message," the text "Welcome to Salesforce Development." appears instantly, with no page reload.

Best Practices

- Name event-handler methods after the action they perform (showMessage), matching the same naming discipline used for Apex methods.
- Use the standard lightning-button component and its variant attribute (brand, success, etc.) rather than a plain HTML <button>, to stay visually consistent with the rest of Salesforce.

Learning Outcome

- Learned event handling.

Task 6 – Dynamic Status Update

Description

Added a status property starting at "Not Applied" and an "Apply Job" button that updates it to "Applied" when clicked.

Why This Task Was Performed

This task builds directly on Task 5's event-handling pattern, but applies it to something more meaningful than a static message: a piece of state that represents a real decision a student would make in the actual application (applying for a job).

Salesforce Concepts Involved

This is the same reactive-property pattern as Task 5, applied to status instead of message. The applyJob() method reassigns this.status, and every part of the template bound to {status} updates immediately and automatically.

```
status = 'Not Applied';

applyJob() {
  this.status = 'Applied';
}
```

What this code does: it initialises status to 'Not Applied', and defines an applyJob() method that changes it to 'Applied' the moment it is called — with no server round-trip and no page refresh.

```
<p><strong>Status :</strong> {status}</p>

<lightning-button
  label="Apply Job"
  variant="success"
  onclick={applyJob}>
```

</lightning-button>

What this code does: it displays the current status value, and renders a green ("success" variant) Apply Job button wired to the applyJob() method, visually signalling to the user that this is a positive, forward-moving action.

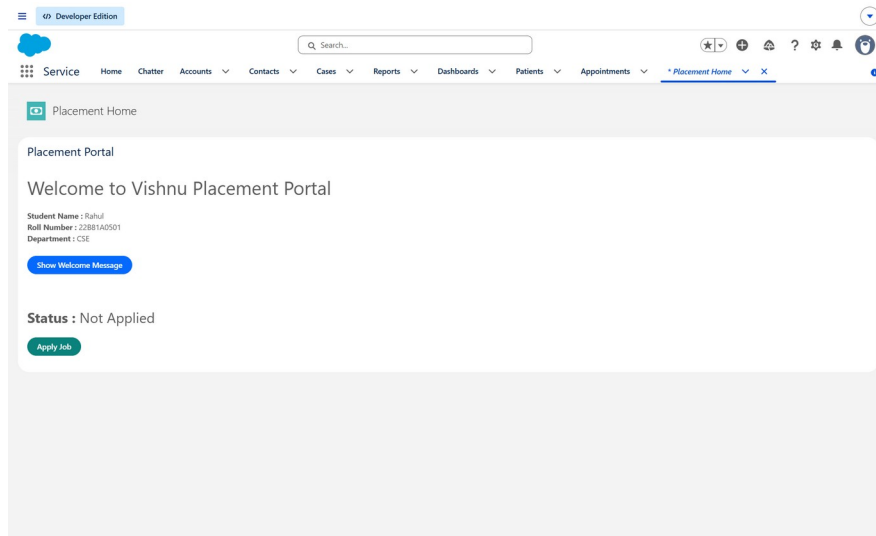


Figure 12: Placement Home Page Showing Status: Not Applied

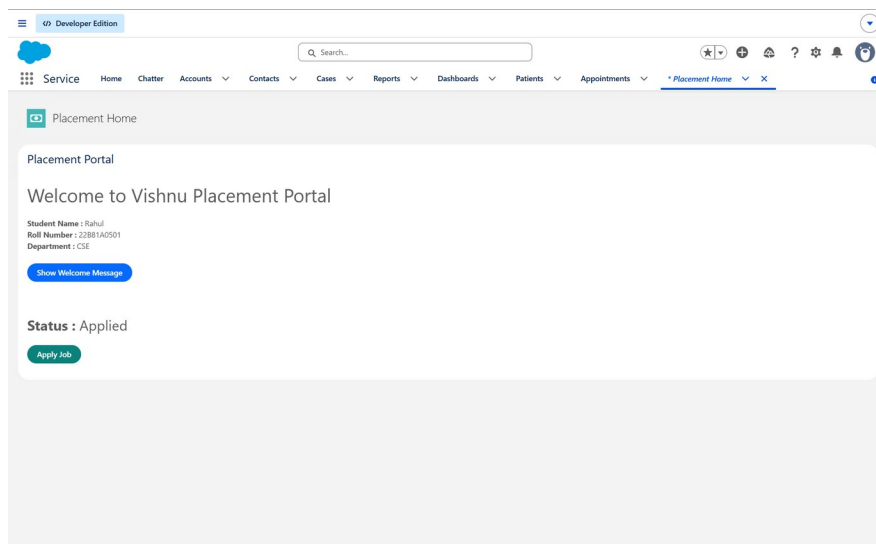


Figure 13: Status Updates to "Applied" Immediately After Clicking Apply Job

Expected Output

The page initially reads "Status : Not Applied." After clicking Apply Job, it updates in place to "Status : Applied," with no page refresh — demonstrating that LWC manages this state entirely on the client side, for now independent of any database record.

Best Practices

- Choose a button variant (success, brand, destructive, etc.) that visually matches the meaning of the action, as done here with success for Apply Job.

- Keep state transitions (Not Applied → Applied) simple and predictable before introducing more complex states.

Learning Outcome

- Learned reactive updates.
-

Task 7 - Mini Project Enhancement

Description

Combined the work from earlier tasks into a small Placement Portal dashboard, adding today's date, number of companies, number of jobs, and applications submitted.

Why This Task Was Performed

Individually, Tasks 3 through 6 each proved one capability in isolation (rendering, binding, click handling, state updates). Task 7 exists to combine all of them into something that actually resembles a feature a user would recognise: a real dashboard summarising the placement season at a glance.

Salesforce Concepts Involved

The today property below uses standard JavaScript — `new Date().toLocaleDateString()` — to compute today's date once, when the component's class is initialised, and format it according to the user's locale.

```
companies = 25;  
jobs = 63;  
applications = 5;  
today = new Date().toLocaleDateString();
```

What this code does: it declares three numeric properties representing dashboard statistics, and a today property computed from the browser's current date, formatted as a locale-appropriate date string (for example, 8/6/2026).

```
<p><strong>Today's Date :</strong> {today}</p>  
<p><strong>Number of Companies :</strong> {companies}</p>  
<p><strong>Number of Jobs :</strong> {jobs}</p>  
<p><strong>Applications Submitted :</strong> {applications}</p>
```

What this code does: it binds each of the four new properties into the template, extending the same welcome card built in Tasks 3–6 into a small dashboard that displays the date alongside placement statistics.

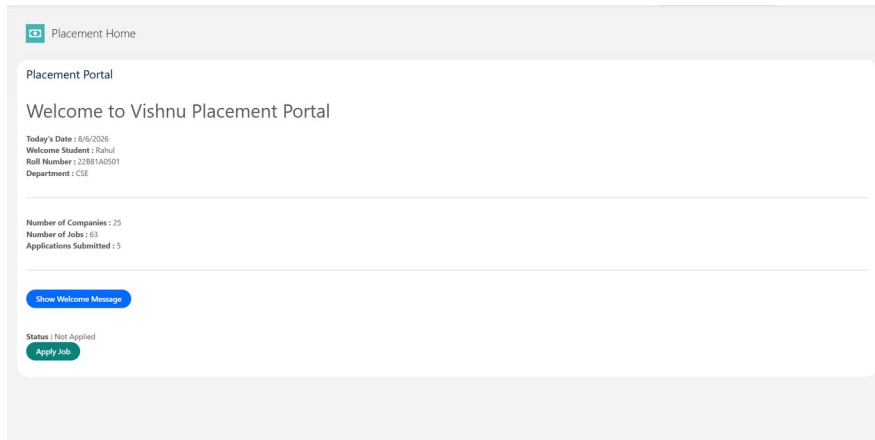


Figure 14: Placement Portal Dashboard — Initial State (Not Applied)

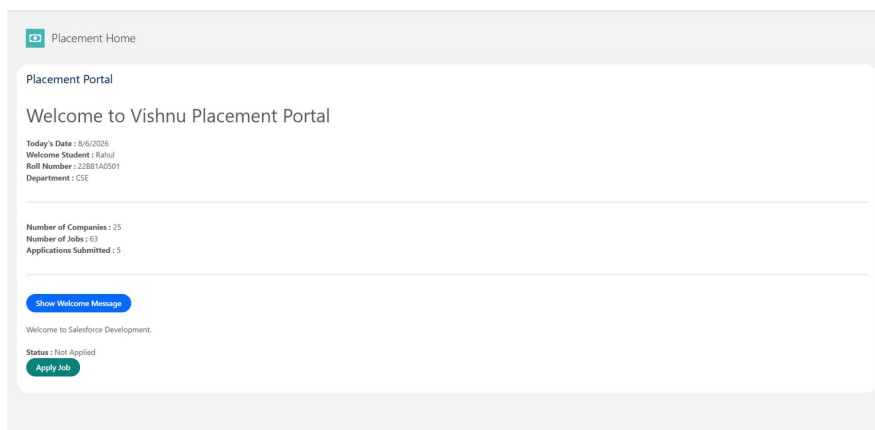


Figure 15: Dashboard After Clicking Show Welcome Message

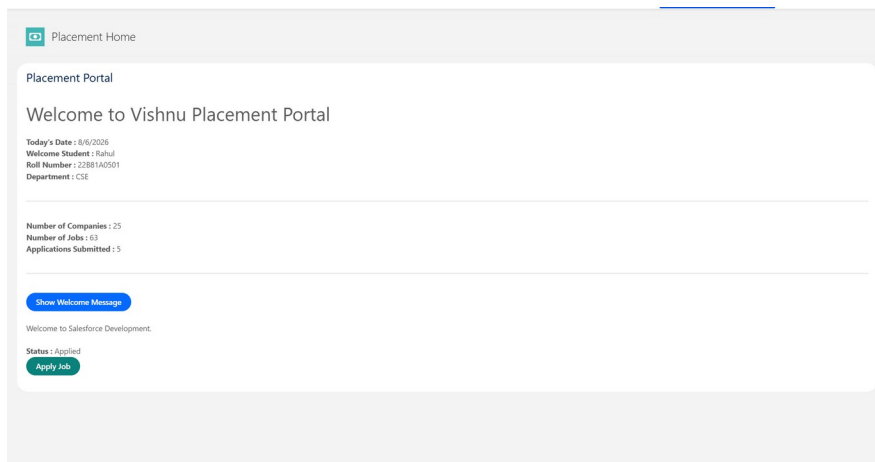


Figure 16: Dashboard After Clicking Apply Job — Status: Applied

Business Purpose

This is the closest Day 4 comes to a feature the Placement Officer or a student would actually use daily: one screen showing the date, key placement statistics, the student's own details, and their application status, with working buttons — everything built so far, working together.

Expected Output

A single component showing today's date, the student's welcome and details, the companies/jobs/applications statistics, the working "Show Welcome Message" button, and the working "Apply Job" status toggle, all on one page.

Best Practices

- Keep hard-coded dashboard numbers (companies, jobs, applications) clearly temporary, since the natural next step is to replace them with values returned from Apex/SOQL.
- Group related properties and related template sections together, so the dashboard stays readable as it grows.

Learning Outcome

- Built the first Placement Portal dashboard.

Task 8 – Understanding Data Binding

Description

A focused, final look at the same binding mechanism used throughout the day, using `studentName` as the example:

```
studentName = 'Rahul';  
  
<p>Hello {studentName}</p>
```

What this code does: it is the smallest possible demonstration of LWC data binding — a single JavaScript property, and a single template expression that displays it. Changing `studentName`'s value, anywhere in the class, automatically updates every place `{studentName}` appears in the template.

Why This Task Was Performed

After building several features that each used data binding for a different purpose (student details, a message, a status), this task steps back to isolate and name the underlying concept clearly: one-way reactive data binding, from the JavaScript class to the template.

Salesforce Concepts Involved

This is described as one-way binding because data flows in a single direction — from the JavaScript property to the displayed HTML. LWC does not offer built-in two-way binding (where a change in the HTML, like typing into a field, automatically updates the JavaScript property); that requires explicitly handling an input event, which is a topic for a later sprint.

The rendered evidence for this exact behaviour is the same shown earlier in Task 4 (see Figure 6), since both tasks exercise the identical `{studentName}` binding.

Learning Outcome

- Changing `studentName` automatically updates the UI.

Challenges Faced

- Learned Salesforce CLI.
- Fixed "Multiple roots found" HTML error.
- Successfully deployed and activated the LWC.

Key Learnings

- HTML creates UI.
- JavaScript controls logic.
- Meta XML exposes the component.
- LWC uses reactive data binding.

Best Practices Followed

- Used meaningful component and property names throughout (placementHome, studentName, showMessage, applyJob).
- Kept HTML (layout) and JavaScript (logic) responsibilities clearly separated.
- Used standard Lightning/SLDS components (lightning-card, lightning-button, SLDS utility classes) instead of custom styling.
- Built incrementally — one capability (render, bind, click, state) proven at a time before combining them into the Task 7 dashboard.

Summary

Successfully created, deployed and activated the placementHome Lightning Web Component. Implemented data binding, event handling, dynamic status updates and a simple Placement Portal dashboard, building a strong foundation for future Apex integration.

Conclusion

Day 4 moved the Placement Management System from a purely backend project into one with a real, working front end. Starting from an empty three-file bundle, the placementHome component grew, task by task, into a small dashboard capable of displaying student and placement data, responding to button clicks, and managing simple state — all using standard web technologies inside Salesforce's Lightning Web Components framework. Every value shown today was hard-coded in JavaScript; the natural and

clearly signposted next step is to replace that hard-coded data with live values retrieved through Apex and SOQL, connecting the UI layer built today to the business logic and data layers built in earlier sprints.